

第十四章：测评系统与竞赛

一、核心概念

1.1 OS 内核竞赛

全国大学生操作系统内核实现赛（OS Comp），要求参赛队伍实现一个能在 RISC-V 平台上运行的 OS 内核，通过官方测试套件的自动化测评。

1.2 测评环境

测评机：

1. 提供 sdcard.img (FAT32, 包含测试程序 ELF)

2. qemu-system-riscv64 -kernel kernel-qemu -drive file=sdcard.img

3. 内核启动 → initcode 遍历测试程序 → fork+exec 运行

4. 捕获串口输出 → 匹配标记字符串 → 计分

标记格式：

OS COMP TEST GROUP START <name> #####
... 测试输出 ...
OS COMP TEST GROUP END <name>

1.3 系统调用兼容性

测试程序用 musl libc 编译，发出 Linux RISC-V ABI 的 syscall。内核必须正确实现这些 syscall 的语义，包括：

- 正确的调用号（Linux RISC-V 号）
- 正确的错误码（负值 errno）
- 正确的边界情况处理

二、测试覆盖

2.1 基础测试（31 项，全部通过）

能力域	测试项	考察点
进程	fork, clone, exit, wait, waitpid, getpid, getppid, yield, execve	进程生命周期
文件	open, openat, close, read, write, fstat, dup, dup2, pipe	基本 I/O
目录	mkdir, unlink, chdir, getcwd, getdents, mount, umount	目录操作
内存	brk, mmap, munmap	堆和映射
时间	gettimeofday, sleep, times	时钟

能力域	测试项	考察点
其他	uname	系统信息

2.2 决赛题目（现场 3 小时）

题目 1：close_range (50分)

```
int close_range(unsigned first, unsigned last, unsigned flags)
```

关闭 [first, last] 范围内的所有 fd。

5 个测试点：基本范围关闭、last=~OU、first>last 静默成功、CLOSE_RANGE_CLOEXEC 标志、稀疏 fd 表。

题目 2：eventfd2 (50分)

```
int eventfd2(unsigned initval, int flags)
```

64 位计数器事件通知。

5 个测试点：基本读写、EFD_SEMAPHORE、EFD_NONBLOCK、累积写、dup/fork 共享。

题目 3：waitid (50分)

```
int waitid(idtype_t idtype, id_t id, siginfo_t *infop, int options)
```

4 个测试点：基本退出等待、WNOHANG、信号杀死、P_PGID 进程组。

附加题：VMA (mmap/munmap/mprotect)

测试 VMA 惰性映射、split、权限修改。

三、关键调试经验

3.1 常见坑

坑	描述	解决
fd 上限	测试打开很多 fd	NOFILE: 16→128
LFN 解析	FAT32 长文件名匹配失败	修复段序号和字符提取
动态链接	musl 程序需要 ld-musl 解释器	PT_INTERP 解析 + fallback
栈大小	测试用栈超 4KB	USERSTACK_PAGES: 1→8
errno 语义	返回值不是负 errno	统一检查
信号杀死	waitid 区分正常退出/被信号杀	exit 时记录退出原因

3.2 如何添加新测试

- 1. 写测试程序 → musl/bare-metal 工具链编译

2. 放入 sdcard 镜像
 3. initcode.c 测试列表中添加路径
 4. `make all` → QEMU 运行 → 观察输出
-

四、八股 / 面试高频问题

Q: 你的内核通过了多少测试? A: 31 项基础 syscall 测试全部通过, 决赛现场 3 小时实现了 `close_range`、`eventfd2`、`waitid` 三个系统调用并通过测试。

Q: 决赛最难的题目是哪个? 怎么做的? A: `waitid` 最复杂——需要区分 `P_PID/P_PGID/P_ALL` 三种等待模式, 区分正常退出和被信号杀死 (`CLD_EXITED` vs `CLD_KILLED`), 实现 `WNOHANG` 非阻塞语义和 `WNOWAIT` 不回收语义。关键是正确填充 `siginfo_t` 结构体。

Q: 比赛中遇到的最棘手的兼容性问题? A: musl 编译的程序做 `fork` 实际发的是 `clone(SIGCHLD, 0, 0, 0, 0)`, 所以 `sys_clone` 必须同时处理线程和 `fork` 语义。另外 musl 的 `open()` 实际走 `openat(AT_FDCWD, ...)`, 内核必须正确处理 `AT_FDCWD`。

五、项目贡献亮点

- 基础 31 项 syscall 测试全部通过, 覆盖进程/文件/目录/内存/时间五大能力域
- 决赛现场 3 小时实现 3 个新系统调用, 体现快速理解规范和底层编码的能力
- 从测试失败到定位根因到修复的完整调试经验 (`fd` 上限、LFN 解析、动态链接等)